Aula 22
Grafos

Conceitos Básicos

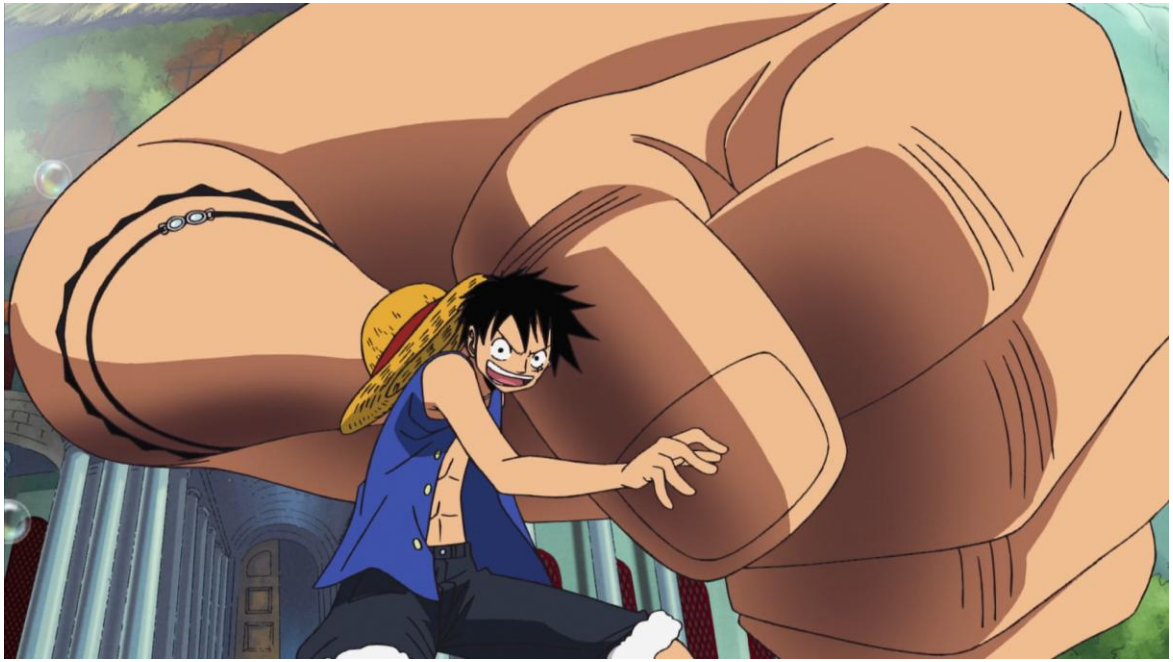
Algoritmos e Estruturas de Dados

Grafos

Conceitos Básicos

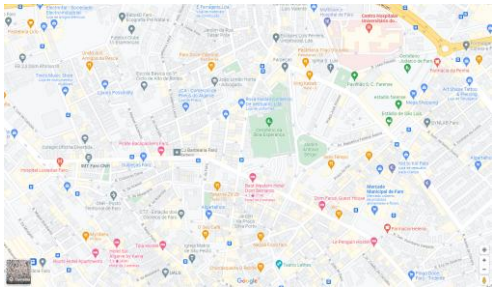
Grafos

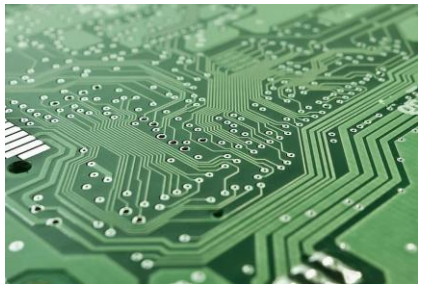
- Grafos podem ser vistos como a evolução seguinte às árvores

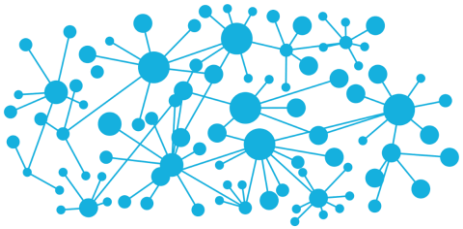


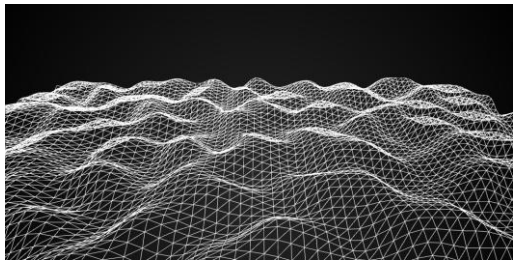
Grafos

- Representação matemática usada para representar itens e ligações entre itens
- Permite representar redes complexas
- Propriedades dos grafos são bastante estudadas e conhecidas
- Inúmeros algoritmos sobre grafos para determinar informação
  - Caminho mais curto
  - Deteção de Ciclos



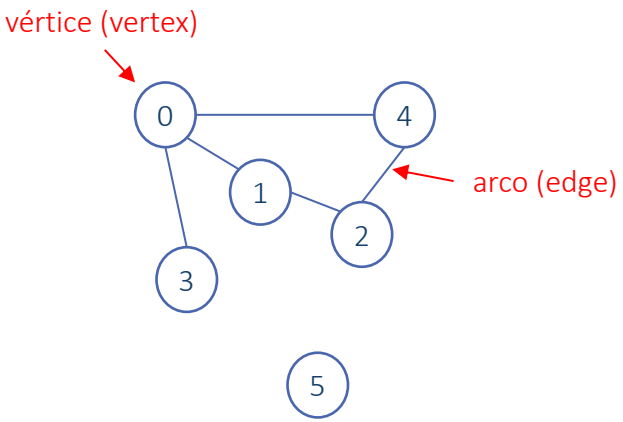






Grafo

- ***Def:***
  - *Um grafo (não dirigido) é um conjunto de vértices e uma coleção de arcos que ligam pares de vértices*



representação visual de um grafo não dirigido

Observação:
Um arco representa uma ligação entre um par de vértices. A partir do vértice 4 posso ir para 2, e a partir de 2 posso ir para 4.

Observação:
Num grafo **não dirigido** os arcos representam ligações nos dois sentidos.

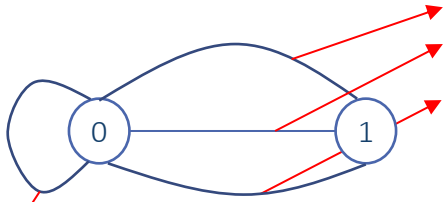
Num grafo **dirigido** os arcos passam a ter um sentido definido, e $arco(a, b) \neq arco(b, a)$

Grafo

- ***Def:***
  - *Um grafo (não dirigido) é um conjunto de vértices e uma coleção de arcos que ligam pares de vértices*

Diz-se uma coleção porque posso ter arcos repetidos. Ex: dois arcos que ligam o mesmo par de vértices (chamam-se arcos paralelos)

Diz-se um conjunto porque não posso ter vértices repetidos num grafo



arcos paralelos para os vértices 0,1

Tb é possível ter um arco de um vértice para ele próprio

Conceitos e definições

Grau: o grau de um vértice corresponde ao número de arcos que referem esse vértice.

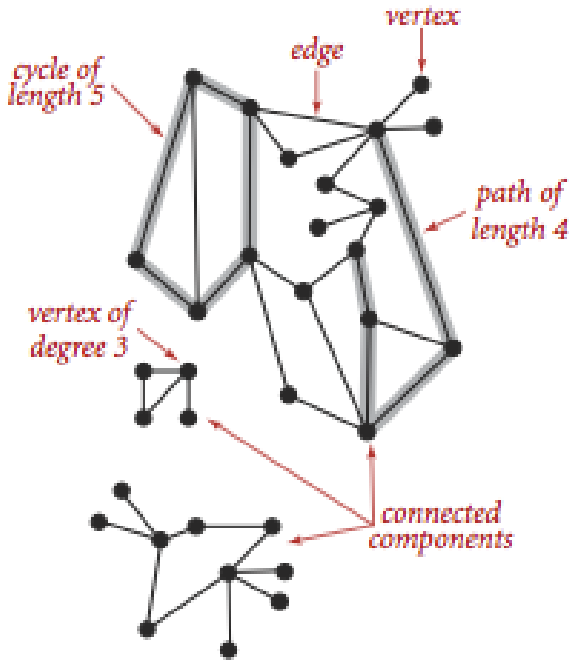
Subgrafo: um subgrafo corresponde a um subconjunto dos arcos de um grafo (e vértices correspondentes) em que o subconjunto constitui um grafo.

Caminho: um caminho num grafo é uma sequência de vértices ligados por arcos, sem arcos repetidos.

Comprimento de um caminho: o tamanho ou comprimento de um caminho é dado pelo número de arcos no caminho.

Ciclo: Um ciclo é um caminho de tamanho >= 1, em que o primeiro e último vértice são o mesmo.

Grafo acíclico: grafo sem ciclos.



Anatomy of a graph

Retirado de:
Sedgewick R. and Wayne K.: Algorithms, 4th edition. Addison-Wesley. 2011.

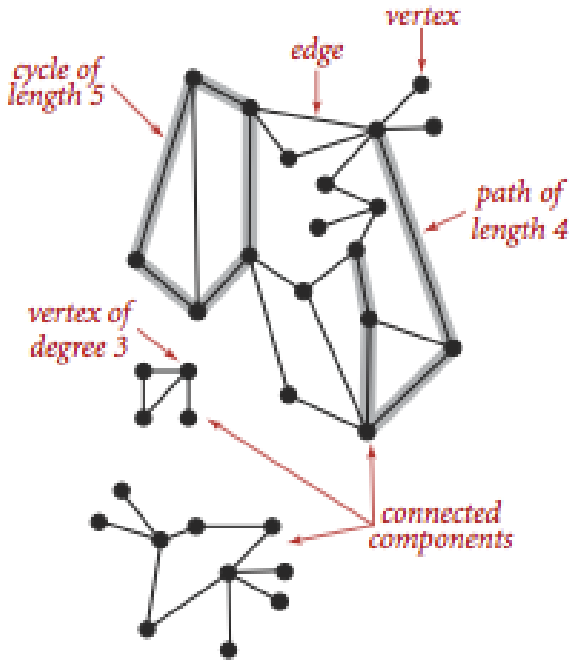
Conceitos e definições

Vértices adjacentes: dois vértices dizem-se adjacentes se existe um arco entre eles.

Vértices ligados: dois vértices dizem-se ligados se existir um caminho que contenha ambos os vértices

Grafo totalmente ligado: um grafo diz-se ligado ou totalmente ligado se existir um caminho para qualquer par de vértices do grafo.

Um grafo não ligado é composto por um conjunto de componentes ligados, que por sua vez são subgrafos totalmente ligados.



Anatomy of a graph

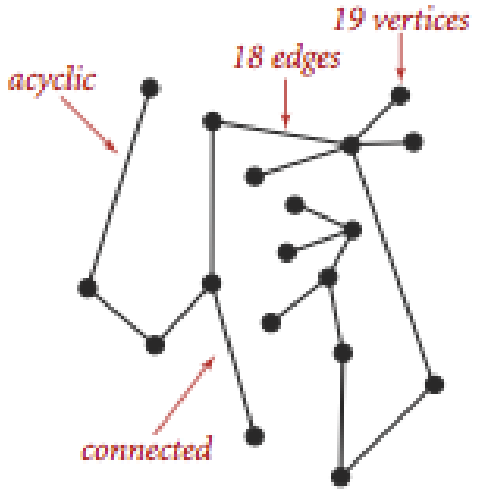
Retirado de:
Sedgewick R. and Wayne K.: Algorithms, 4th edition. Addison-Wesley. 2011.

Árvores como Grafos

- *Def:*
  - *Uma árvore é um grafo não dirigido acíclico totalmente ligado*

Observação:
Mais especificamente, um grafo G com V vértices é uma árvore sse:

- G tem V-1 arcos e não tem ciclos
- G é totalmente ligado
- Remover algum dos arcos faz com que G não seja ligado
- Adicionar um novo arco irá criar um ciclo
- Cada par de vértices em G é ligado por apenas um caminho



A tree

Retirado de:
Sedgewick R. and Wayne K.: Algorithms, 4th edition. Addison-Wesley. 2011.

Grafos

Implícitos

Grafo Implícito

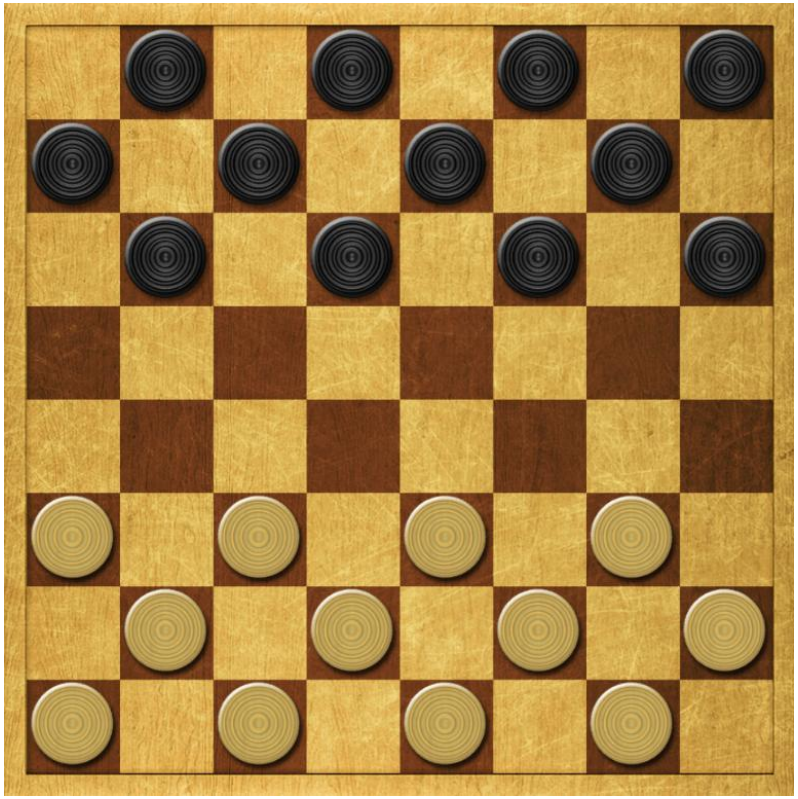
- Um grafo implícito corresponde a um problema ou conjunto de informação onde
  - A informação tem as propriedades de um grafo
  - Tenho estados/itens/posições, ou qualquer outro bocado de informação que corresponde a um vértice
  - A partir de cada estado/item/posição, consigo determinar quais os seus vizinhos/adjacentes

- Num grafo implícito, os vizinhos são obtidos através de uma função
- Normalmente existe um padrão bem definido para determinar os vizinhos

Exemplo Grafo Implícito

As casas de um tabuleiro de damas podem ser vistas como um grafo implícito

$$vizinhos\big((l,c)\big) = \left\{ \begin{matrix} (l-1,c-1),(l-1,c),(l-1,c+1) \\ (l,c-1),(l,c+1), \\ (l+1,c-1),(l+1,c)(l+1,c+1) \end{matrix} \right\},$$

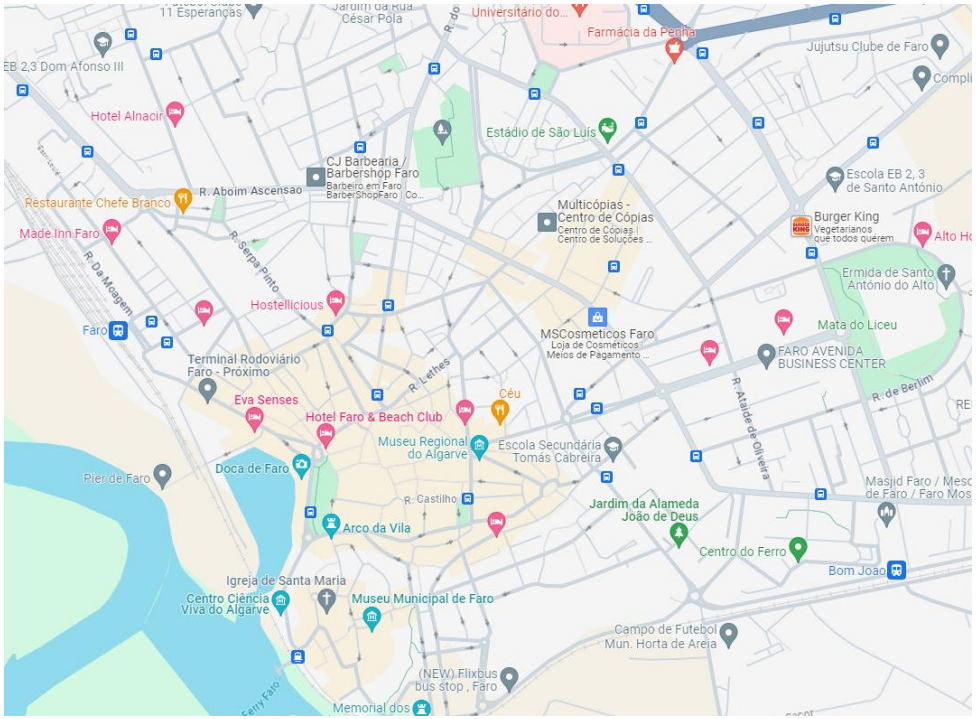


Grafos

Como estrutura de dados

Grafos como estrutura de dados

- Em determinados tipos de problemas, não é possível usarmos um grafo implícito
  - Por exemplo, quando não existe um padrão óbvio para determinar os vizinhos



Grafos como estrutura de dados

- Adicionalmente, usar uma representação explícita permite-nos
  - Compreender melhor as propriedades de um grafo
  - Estudar mais facilmente a sua complexidade

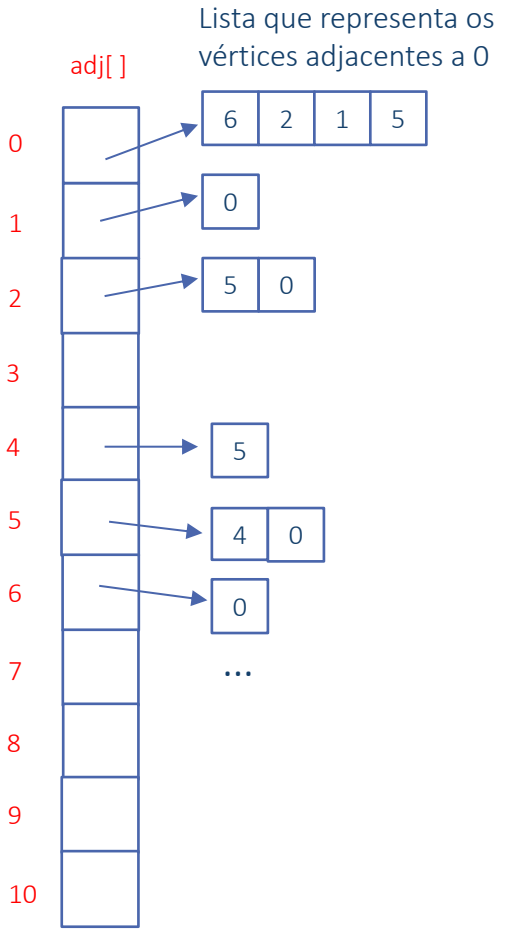
Especificação Classe Grafo

public Class Graph		
	`Graph(int v)`	Cria um grafo com v vértices (numerados de 0 a v-1) e sem arcos.
`int`	`vCount()`	Retorna o número de vértices no grafo.
`int`	`eCount()`	Retorna o número de arcos (edges) no grafo
`void`	`addEdge(int v1, int v2)`	Adiciona um arco entre o vértice v1 e o vértice v2
`Iterable<Integer>`	`adj(int v)`	Retorna um iterador que permite iterar sobre todos os vertices adjacentes ao vértice v recebido
`int`	`degree(int v)`	Retorna o grau de um vértice, ou seja o número de arcos associados ao vértice
`int`	`maxDegree()`	Retorna o maior grau do Grafo.
`String`	`toString()`	Retorna uma representação em String do Grafo
`Static Graph`	`parse(Scanner sc)`	Faz parse de um grafo a partir de um scanner (por exemplo criado para ler um ficheiro), devolvendo o Grafo lido.

Observação:
Esta implementação é uma implementação muito simples, em que não guardamos mais informação do que o índice do vértice.
No entanto, é fácil acrescentar um objeto genérico a todos os vértices do grafo.

Implementação Tipo Grafo

- O tipo Grafo pode ser implementado de forma eficiente
  - Usando um *array* de listas de adjacências



Observação 1:
Se quisermos guardar objetos dentro dos vértices de um grafo podemos criar uma classe nó com o objeto e uma lista de adjacências.

Cria-se um array destes nós.

Observação 2:
Esta implementação só funciona quando o número de vértices de um grafo é conhecido à partida, e não aumentar com muita frequência.

Se o número de vértices/nós de um grafo for muito dinâmico, então é preferível representar um grafo como um nó com arcos ou ponteiros para outros nós.

Implementação Tipo Grafo

```
package aed.graphs;

import java.util.LinkedList;
import java.util.Scanner;

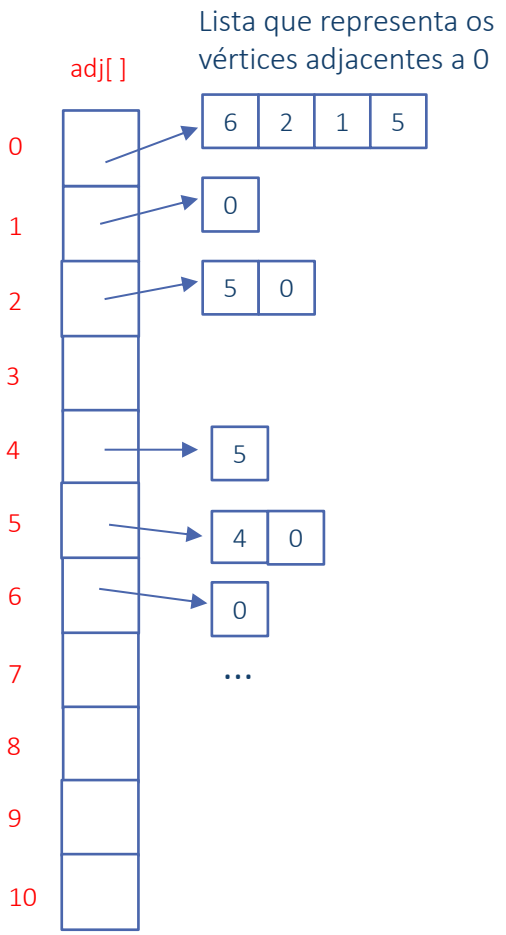
public class Graph {

	private int vCount;
	private int eCount;
	private LinkedList<Integer>[] adj;

	@SuppressWarnings("unchecked")
	public Graph(int vCount)
	{
		this.vCount = vCount;
		this.eCount = 0;
		this.adj = (LinkedList<Integer>[]) new Object[vCount];

		//Initialise all adjacency lists
		for(int i = 0 ; i < vCount; i++)
		{
			this.adj[i] = new LinkedList<Integer>();
		}
	}
```

Podiamos substituir isto por um ArrayList sem qualquer problema



Implementação Tipo Grafo

```
public int vCount() { return this.vCount;}

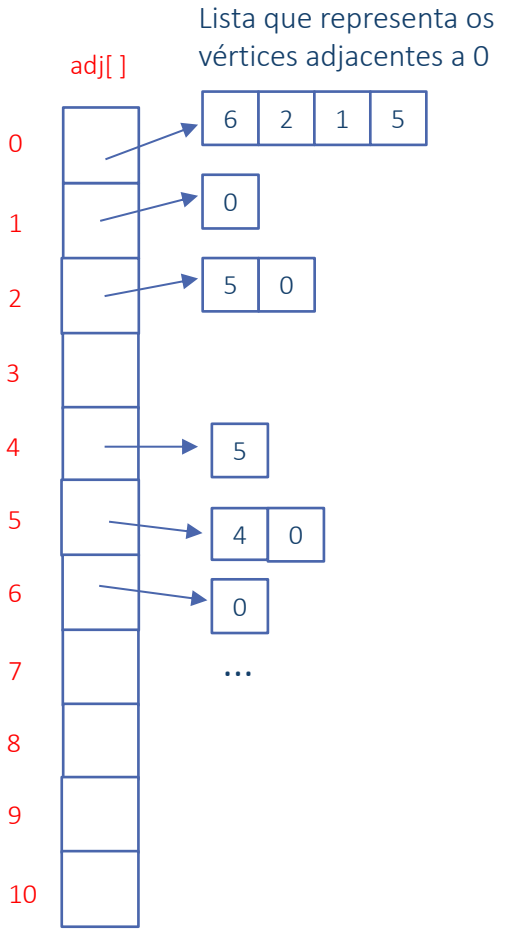
public int eCount() { return this.eCount;}

public void addEdge(int v1, int v2)
{
      this.adj[v1].add(v2);
      this.adj[v2].add(v1);
      this.eCount++;
}

public int degree(int v) { return this.adj[v].size();}

public int maxDegree()
{
      int max = 0;
      int degree;
      for(int i = 0; i < this.vCount ; i++)
      {
            degree = degree(i);
            if(degree > max) max = degree;
      }
      return max;
}

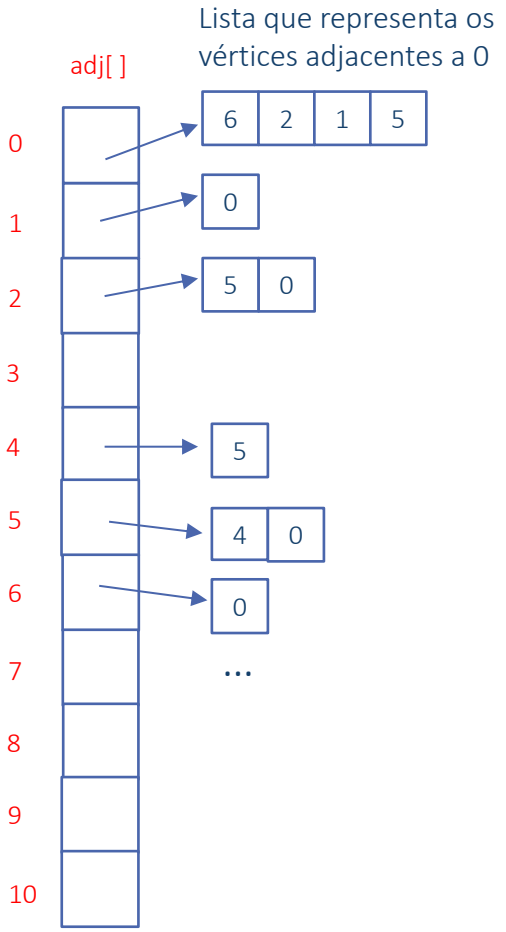
public Iterable<Integer> adj(int v) {return this.adj[v];}
```



Implementação Tipo Grafo

```
public static Graph parse(Scanner sc)
{
      Graph g = new Graph(sc.nextInt());
      int v1,v2;
      g.eCount = sc.nextInt();

      for(int i = 0; i < g.eCount; i++)
      {
            v1 = sc.nextInt();
            v2 = sc.nextInt();
            g.addEdge(v1,v2);
      }
      return g;
}
```



Complexidade Temporal

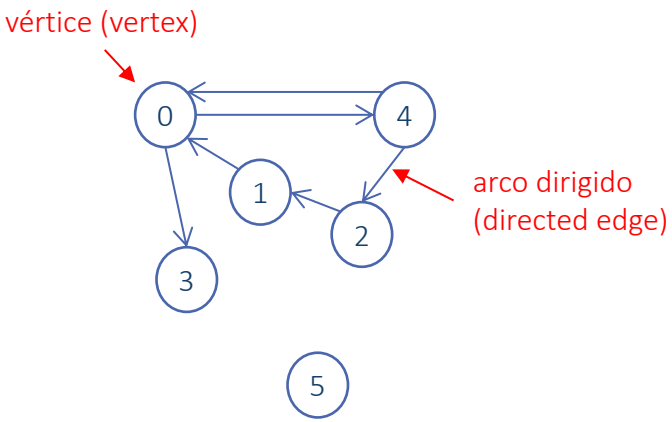
	Caso médio	Pior caso
addEdge	O(1)	O(1)
Iterar sobre vertices adjacentes a v	O(grau(v))	O(maxgrau())

Grafos

Dirigidos

Grafo dirigido

- ***Def:***
  - *Um grafo dirigido (directed graph) é um conjunto de vértices e uma coleção de arcos dirigidos. Um arco dirigido liga um par ordenado de vértices.*



representação visual de um grafo dirigido

Observação:
Um arco dirigido representa uma ligação entre o 1.º elemento e o 2.º de um par de vértices. A partir do vértice 4 posso ir para 2, mas a partir de 2 não posso ir para 4.

Observação:
Num grafo **não dirigido** os arcos representam ligações nos dois sentidos.

Num grafo **dirigido** os arcos passam a ter um sentido definido, e $arco(a, b) \neq arco(b, a)$

Conceitos e definições Grafos Dirigidos

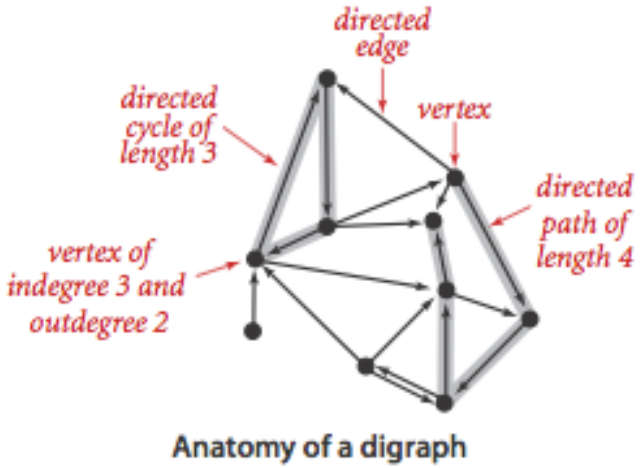
Grau: o grau de um vértice corresponde ao número de arcos que saem desse vértice.

Caminho dirigido: um caminho num grafo dirigido é uma sequência de vértices, em que cada vértice tem um arco dirigido para o seu sucessor, e não existem arcos repetidos.

Comprimento de um caminho: o tamanho ou comprimento de um caminho dirigido é dado pelo número de arcos no caminho.

Ciclo dirigido: Um ciclo dirigido é um caminho dirigido de tamanho >= 1, em que o primeiro e último vértice são o mesmo.

Grafo acíclico dirigido (DAG): grafo dirigido sem ciclos dirigidos.



Anatomy of a digraph

Retirado de:
Sedgewick R. and Wayne K.: Algorithms, 4th edition. Addison-Wesley. 2011.

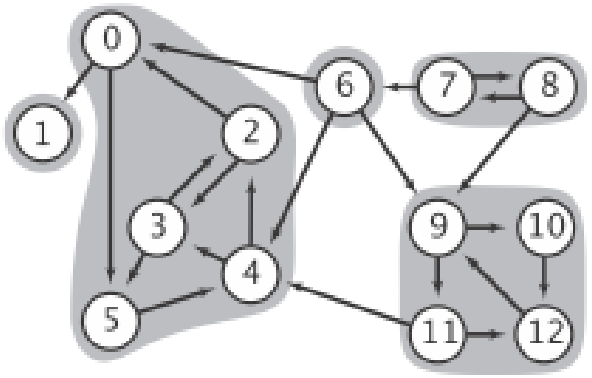
Conceitos e definições Grafos Dirigidos

Vértice alcançável: Um vértice w diz-se alcançável a partir de um vértice v, se existir um caminho dirigido de v para w.

Vértices fortemente ligados: dois vértices v e w dizem-se fortemente ligados sse forem mutuamente alcançáveis. Ou seja, existe um caminho dirigido de v para w, e um caminho dirigido de w para v.

Grafo fortemente ligado: um grafo dirigido diz-se fortemente ligado se todos os seus vértices são fortemente ligados entre si.

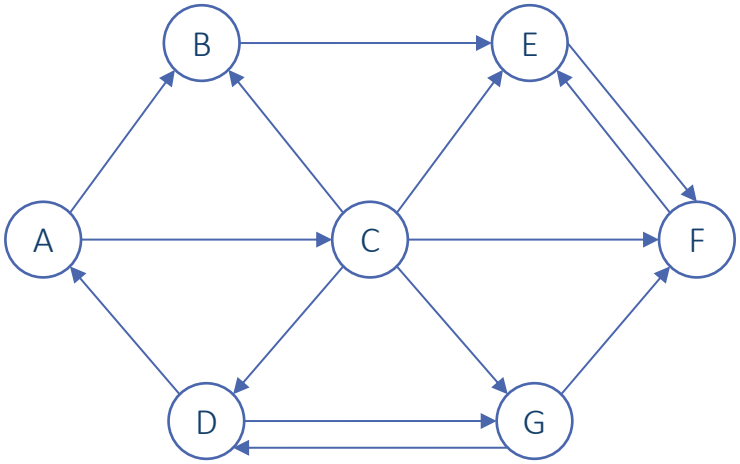
Um grafo dirigido que não seja fortemente ligado é composto por um conjunto de componentes fortemente ligados, que por sua vez são subgrafos dirigidos fortemente ligados.



A digraph and its strong components

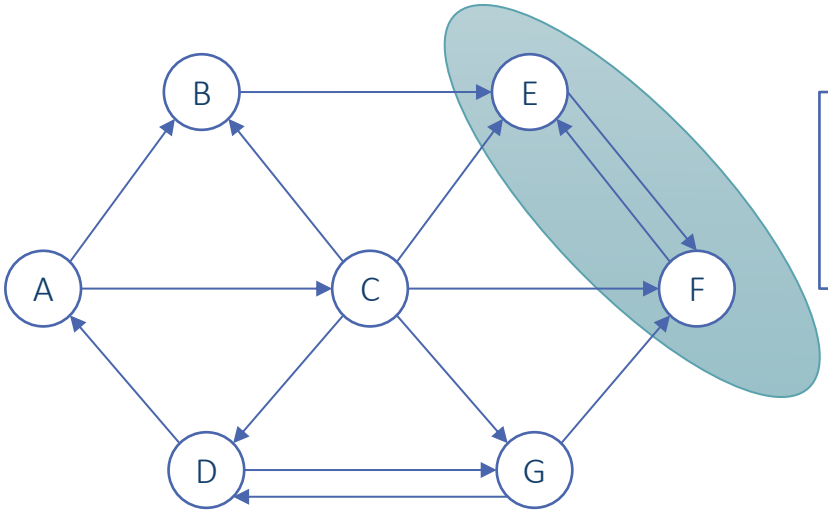
Retirado de:
Sedgewick R. and Wayne K.: Algorithms, 4th edition. Addison-Wesley. 2011.

Exemplo



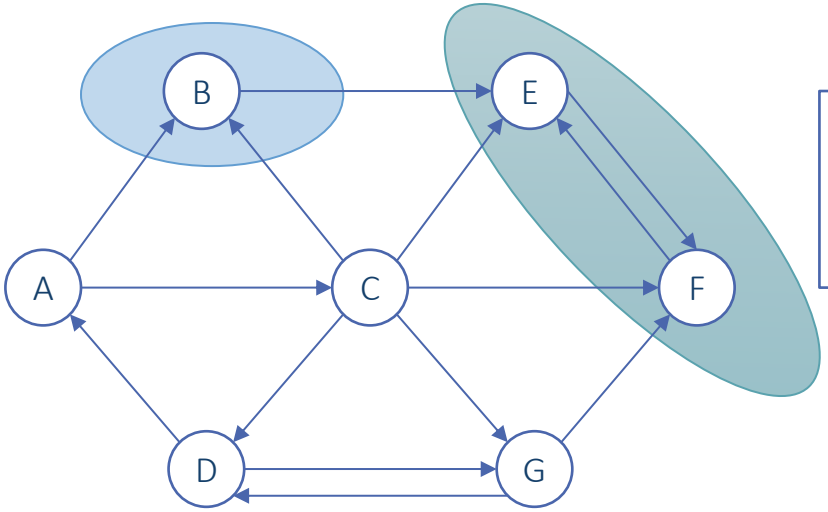
Indique o número de componentes fortemente ligados do grafo

Exemplo



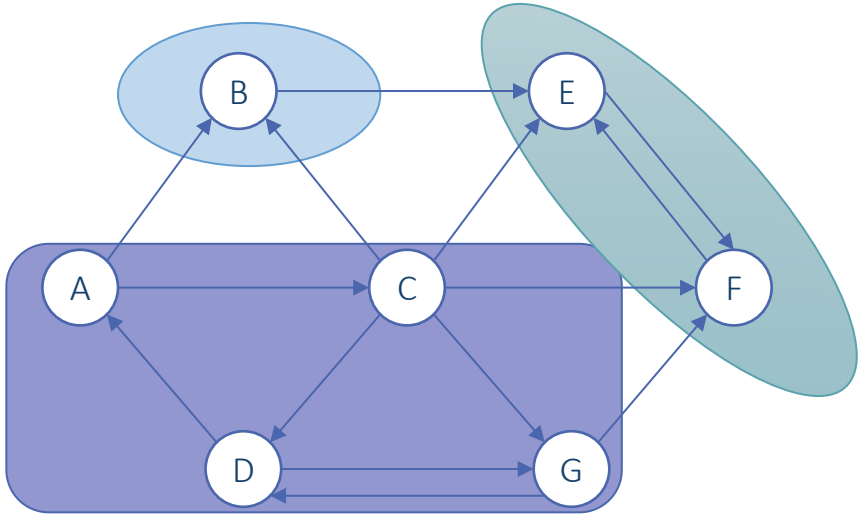
Indique o número de componentes fortemente ligados do grafo

Exemplo



Indique o número de componentes fortemente ligados do grafo

Exemplo



Indique o número de componentes fortemente ligados do grafo

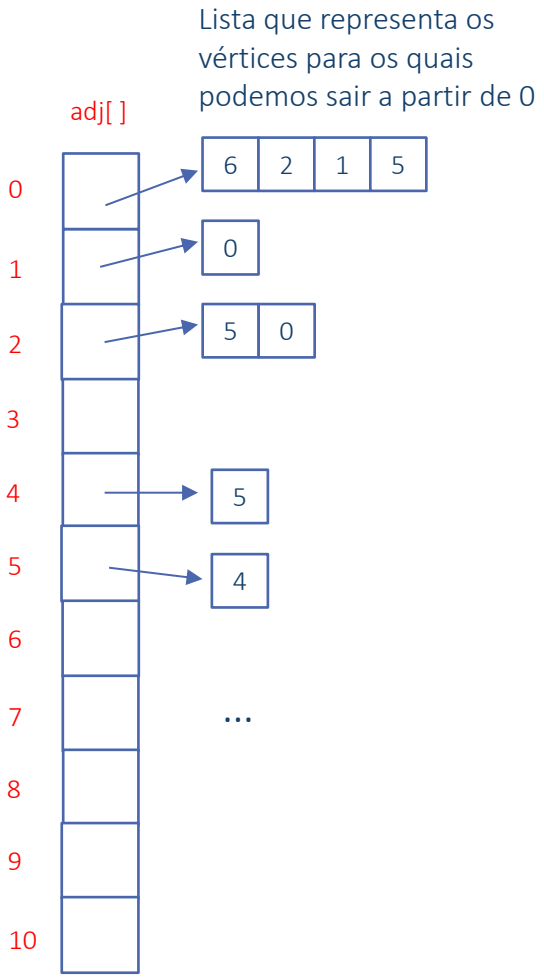
Resposta: 3

Especificação Classe Grafo Dirigido

public Class DiGraph		
	`DiGraph(int v)`	Cria um grafo dirigido com v vértices (numerados de 0 a v-1) e sem arcos.
`int`	`vCount()`	Retorna o número de vértices no grafo.
`int`	`eCount()`	Retorna o número de arcos (edges) no grafo
`void`	`addEdge(int v1, int v2)`	Adiciona um arco dirigido v1 -> v2 ao grafo
`Iterable<Integer>`	`adj(int v)`	Retorna um iterador que permite iterar sobre todos os vertices adjacentes para os quais podemos ir a partir de v
`int`	`degree(int v)`	Retorna o grau de um vértice, ou seja o número de arcos que partem deste vértice
`int`	`maxDegree()`	Retorna o maior grau do Grafo.
`Digraph`	`reverse()`	Retorna um novo grafo dirigido que é uma cópia do grafo, mas com todos os arcos trocados
`String`	`toString()`	Retorna uma representação em String do Grafo
`Static DiGraph`	`parse(Scanner sc)`	Faz parse de um grafo a partir de um scanner (por exemplo criado para ler um ficheiro), devolvendo o Grafo lido.

Implementação Tipo Grafo Dirigido

- O tipo Grafo Dirigido pode ser implementado da mesma forma que um Grafo



Observação:

Apenas guardamos na lista de adjacências os vértices para os quais podemos sair.

Implementação Tipo Grafo Dirigido

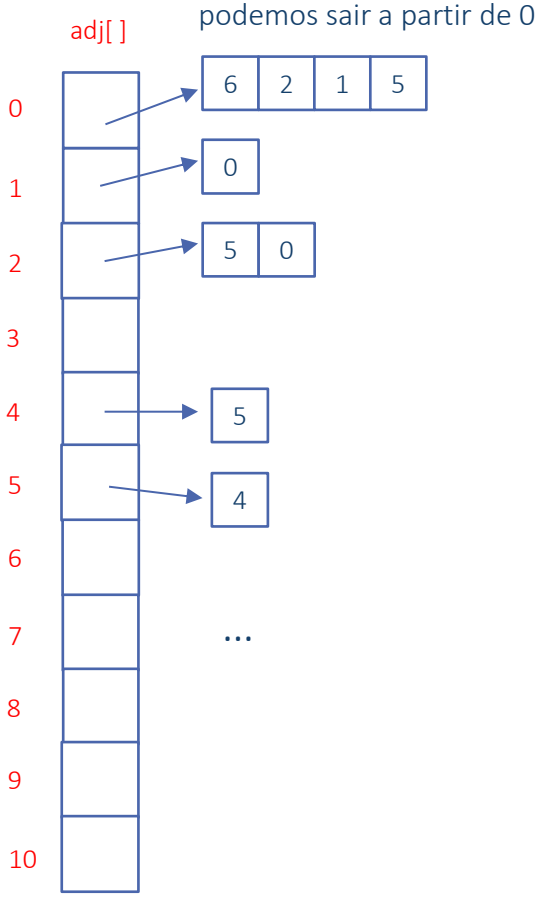
```
public class DiGraph {

      private int vCount;
      private int eCount;
      private LinkedList<Integer>[] adj;
      …

      public void addEdge(int v1, int v2)
      {
            this.adj[v1].add(v2);
            this.eCount++;
      }
      …

      public DiGraph reverse()
      {
            DiGraph reverse = new DiGraph(this.vCount);
            for(int v1 = 0; v1 < this.vCount; v1++)
            {
                  for(int v2 : this.adj[v1])
                  {
                        reverse.addEdge(v2, v1);
                  }
            }
            return reverse;
      }
```

Lista que representa os vértices para os quais podemos sair a partir de 0



Observação:
Os restantes métodos são iguais à classe Grafo

Complexidade Temporal

	Caso médio	Pior caso
addEdge	O(1)	O(1)
Iterar sobre vertices adjacentes a v	O(grau(v))	O(maxgrau())